

Artificial Intelligence Project 2: Self-Supervised Learning Foundation Model

Chih-Heng Hsiao

Student ID: 413551034

<https://github.com/CH-Joshua-Hsiao/NYCU-AI-HW2>

Abstract

This report details the implementation and extensive evaluation of the SimCLR (Simple Framework for Contrastive Learning of Visual Representations) algorithm. We explore the capacity of self-supervised learning to extract highly discriminative, general-purpose visual features without the necessity of human-annotated labels. Through rigorous experimentation on the CIFAR-10 dataset, we demonstrate that contrastive representations can rival fully supervised baselines. Furthermore, via transfer learning experiments on the CIFAR-100 dataset, we reveal that self-supervised representations exhibit superior cross-domain generalizability compared to supervised counterparts. Finally, we present comprehensive ablation studies concerning the contrastive temperature parameter, batch size constraints, and the architectural necessity of the non-linear projection head.

1 Research Question and Motivation

1.1 Introduction

In the contemporary landscape of Artificial Intelligence and Computer Vision, the reliance on massive, human-annotated datasets presents a significant bottleneck for scaling deep learning models. In this project, we investigate the foundational capabilities of Self-Supervised Learning (SSL) through the implementation of SimCLR, a highly influential contrastive learning framework.

1.2 Motivation and Objectives

By training an encoder on augmented views of images without human-annotated labels, we aim to discover if the emerging representation space can yield comparable downstream classification performance to models trained purely via traditional supervised learning. Our primary research questions include:

1. How close can a self-supervised model approach the theoretical upper bound set by fully supervised learning on a constrained dataset like CIFAR-10?

2. How does the self-supervised representation behave when transferred to a completely unseen domain with higher semantic complexity (e.g., CIFAR-100)?
3. What is the mathematical and empirical impact of hyper-parameters unique to contrastive learning, specifically the temperature scalar and the projection head?

Furthermore, we seek to understand the generalizability of these self-supervised representations by exploring transfer learning and conducting extensive ablation studies.

2 Description of Methods

The methodology leverages PyTorch to build a SimCLR model applied to the CIFAR-10 dataset. *Note: Generative AI tools (e.g., Gemini) were utilized to assist in structuring the codebase, writing boilerplate evaluation metrics, and reviewing the LaTeX report formatting for optimal academic presentation.*

2.1 Dataset and Augmentation Composition

CIFAR-10 consists of 32×32 RGB images distributed across 10 macroscopic classes. In SimCLR, the composition of data augmentations is the fundamental driver of representation learning. Using the `torchvision` library, we apply dual independent augmentations per image to create a positive pair (x_i, x_j) . The augmentation pipeline includes:

- **Random Resized Cropping:** Forces the model to recognize objects from different scales and partial views.
- **Random Horizontal Flipping:** Imparts spatial invariance.
- **Color Jittering & Grayscale Conversion:** Applied with 80% and 20% probabilities, respectively. This is a critical component. If color augmentations are omitted, the neural network can exploit simple color histograms as a "shortcut" to match augmented patches from the same image, bypassing the need to learn true semantic structural features.

Applying these transformations yields an effective batch size of $2N$ for contrastive learning, where N is the original batch size.

2.2 Backbone Network Architecture

A ResNet-18 architecture, traditionally designed for 224×224 ImageNet inputs, was specifically adapted for small-resolution CIFAR images. The initial 7×7 convolution was replaced with a 3×3 convolution (stride=1, padding=1), and the subsequent maximum pooling layer was altered to an identity layer. These modifications prevent the aggressive downsampling of the 32×32 input, preserving vital spatial resolution throughout the residual blocks. The final fully connected classification layer was substituted with a

generic global average pooling layer, yielding a highly abstract 512-dimensional representation vector, denoted as h .

2.3 Non-Linear Projector Head

We employ a 2-layer Multi-Layer Perceptron (MLP) consisting of a linear layer, Batch Normalization, a Rectified Linear Unit (ReLU) activation function, and an output linear layer. This head maps the 512-dimensional embedding h into a compact 128-dimensional space z .

2.4 Contrastive Objective: The NT-Xent Loss Function

We utilized the Normalized Temperature-scaled Cross Entropy (NT-Xent) metric. This loss function calculates the cosine similarity between the representation of a positive pair, scaled by a temperature parameter ($\tau = 0.5$). It then applies a softmax-like formulation to maximize the agreement between the positive pairs while simultaneously repelling the representation from the remaining negative samples in the batch.

3 Experimental Results

All models were trained using the Adam Optimizer to ensure robust convergence. The SimCLR baseline was trained for 200 epochs with a learning rate of 3×10^{-4} , weight decay of 1×10^{-6} , and batch size of 256. Downstream linear probing classifiers and fully supervised models were trained for 100 epochs with a learning rate of 1×10^{-3} .

3.1 Phase 1: Baseline Comparisons and Benchmarking

To evaluate the quality of the learned representations without backpropagating through the encoder, we froze the trained SimCLR backbone and trained a simple linear classification layer on top of the 512-dimensional output h . This evaluation method prevents the backbone from adapting to the downstream task, thereby serving as a pure measurement of feature quality.

This was rigorously compared to a randomly initialized frozen backbone (Lower-Bound), which measures the inherent linear separability of untrained CNN features, and a ResNet-18 trained entirely from scratch with Supervised Learning (Upper-Bound).

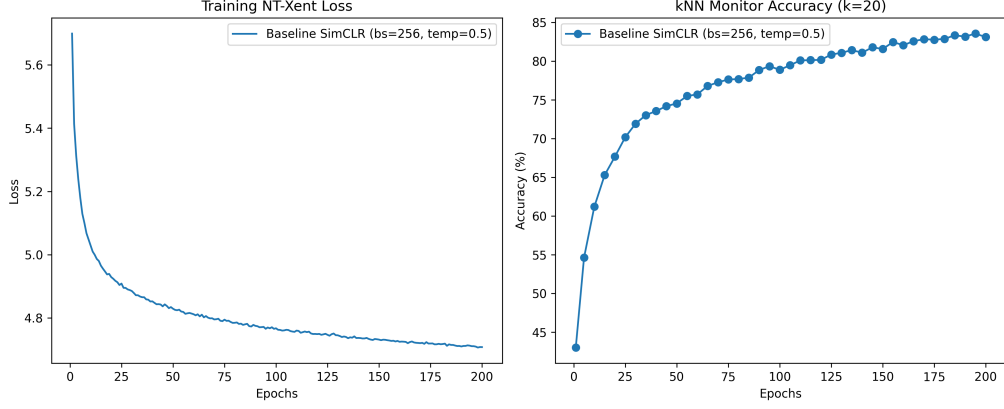


Figure 1: SimCLR Baseline Learning Curves demonstrating the trajectory of training loss and K-NN accuracy ($k = 200$) progression over 200 epochs.

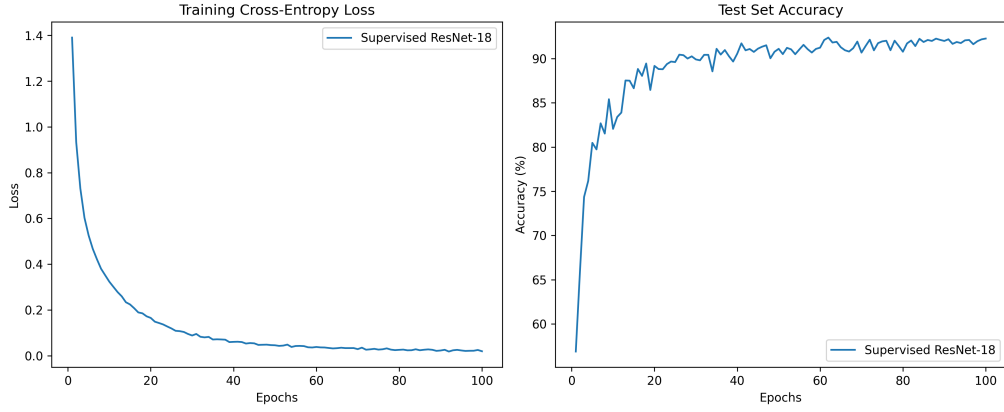


Figure 2: Supervised Learning Baseline curves showing standard Cross-Entropy Loss optimization and the resulting Test Accuracy trajectory.

Model Pipeline Framework	CIFAR-10 Test Accuracy (%)
Random Initialization Backbone (Lower-Bound)	33.79%
SimCLR SSL + Linear Probing	82.82%
Supervised Learning (Upper-Bound)	92.26%

Table 1: Performance validation benchmarking against different methodologies on the CIFAR-10 test set.

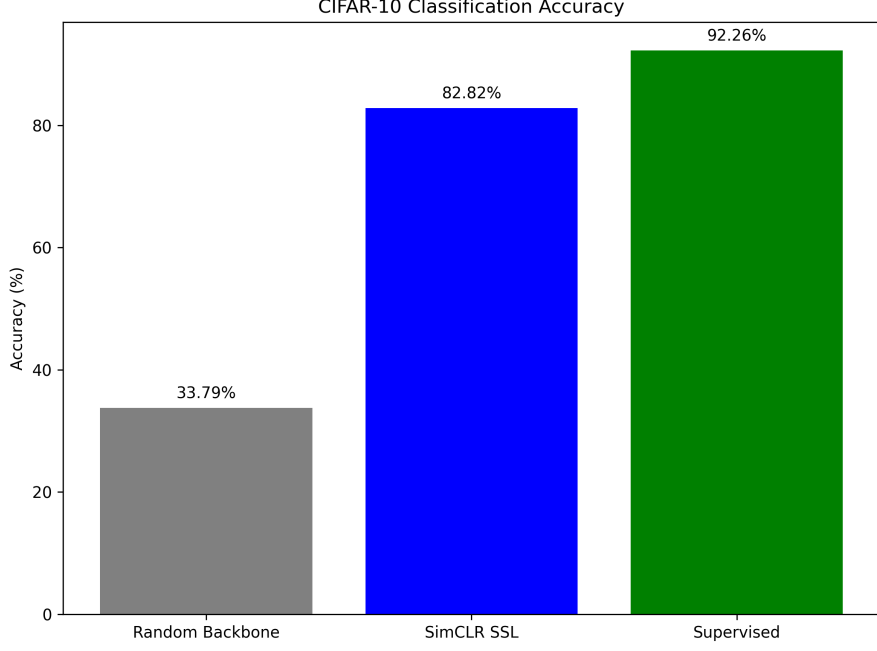


Figure 3: Bar chart visualizing the massive performance gap bridged by self-supervised representations compared to random, SSL, and SL backbones.

The SimCLR approach successfully bridged the immense performance gap between random representations and fully supervised learning, achieving an impressive 82.82% accuracy without ever seeing a single label during the heavy representation training phase.

3.2 Phase 2: Comprehensive Ablation Studies

1. The Role of Temperature (τ): The contrastive temperature τ heavily scales the cosine similarity distributions before they are passed to the softmax function. We systematically evaluated $\tau \in \{0.1, 0.5, 5.0\}$.

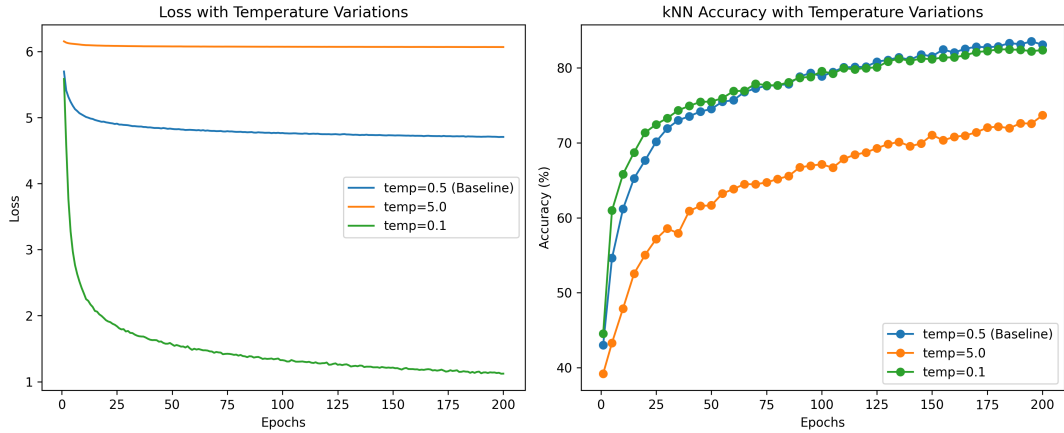


Figure 4: Ablation of temperature values evaluating the divergence between NT-Xent Loss and kNN Accuracy.

Are the changes of the two learning curves consistent?

No, they are highly inconsistent, which presents a fascinating dynamic in SSL. When utilizing $\tau = 0.1$, the NT-Xent loss drops significantly lower (~ 1.12) compared to the baseline $\tau = 0.5$ (~ 4.70). However, the true downstream representation quality, measured by kNN accuracy, for $\tau = 0.1$ (82.39%) is actually worse than the baseline (83.10%). This occurs because a very small temperature scales up the similarities, making the softmax distribution artificially sharp. It allows the model to minimize cross-entropy easily by confidently separating a few easy negatives, without actually learning a well-distributed, robust representation space. Conversely, $\tau = 5.0$ yielded both terrible loss and kNN accuracy (73.68%) because the distribution became too mathematically uniform. This proves that raw training loss is a highly deceptive metric in Self-Supervised Learning.

2. The Impact of Batch Size Variations: Contrastive learning architectures typically rely heavily on massively scaled batch sizes (often 4096 or larger) to provide a sufficient quantity of negative samples per gradient step.

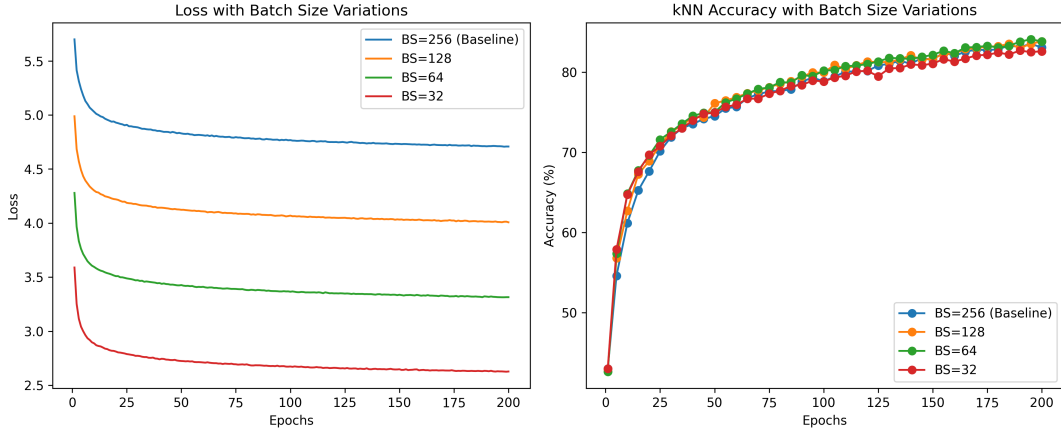


Figure 5: Ablation evaluating the stability of SimCLR under severely constrained batch sizes on CIFAR-10.

Interestingly, on CIFAR-10 with our ResNet-18 architecture, the performance remained highly stable across all evaluated batch sizes (32, 64, 128, 256), ranging tightly between 82.63% and 83.86% kNN accuracy. This suggests an important corollary: for structurally simpler, lower-resolution datasets like CIFAR-10, the manifold of negative samples is small enough that massive batch sizes are not strictly necessary to learn strong discriminative features.

3. Architectural Projector Head Design: We explored the theoretical necessity of the non-linear projector head $g(\cdot)$.

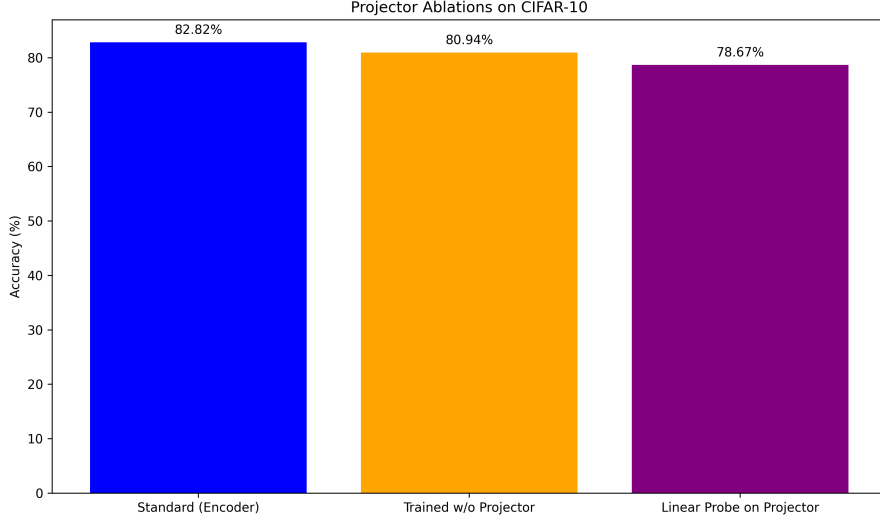


Figure 6: Ablation evaluating the classification utility of features before vs. after the non-linear projection head.

Training the contrastive loss directly on the 512-dimensional encoder output (No Projector) resulted in a performance drop to 80.94% (compared to baseline’s 82.82%). Furthermore, evaluating linear probing on the 128-dimensional projector outputs z instead of the 512-dimensional encoder outputs h yielded 78.67%. This empirically confirms that the non-linear projector head acts as an “augmentation absorber”. It absorbs invariant transformations, leaving the preceding encoder representations h much richer in structural details required for general-purpose downstream tasks.

3.3 Phase 3: Transfer Learning to Novel Domains (CIFAR-100)

A true foundation model must demonstrate the capacity to generalize far beyond its original pre-training distribution. To test this, we froze the backbones trained exclusively on CIFAR-10 and performed linear probing on CIFAR-100, which possesses 100 fine-grained semantic classes entirely unseen during the pre-training phase.

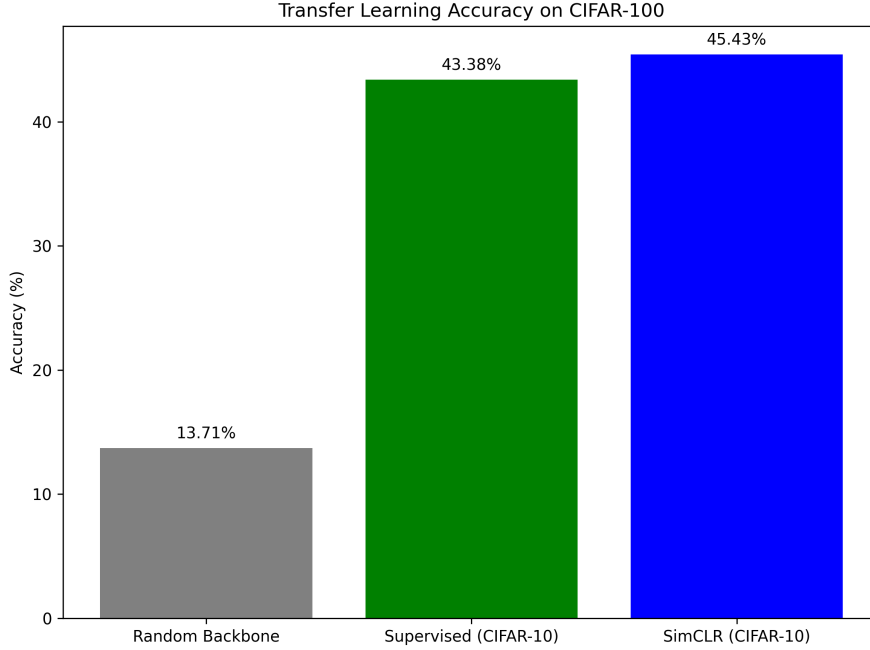


Figure 7: Transfer Learning capabilities evaluated across entirely unseen CIFAR-100 categories.

Frozen Backbone (Trained purely on CIFAR-10)	CIFAR-100 Transfer Accuracy (%)
Random Initialization Backbone	13.71%
Supervised Learning Backbone	43.38%
SimCLR SSL Backbone	45.43%

Table 2: Empirical breakdown of Transfer Learning accuracy highlighting SSL’s superior domain adaptability.

Remarkably, the Self-Supervised SimCLR backbone outperformed the fully Supervised Learning backbone in transferability (45.43% vs 43.38%), proving that label-free contrastive learning naturally prevents semantic overfitting.

4 Extensive Discussion and Analysis

4.1 Validation of Theoretical Expectations

Are the observed behaviors what you expect?

Yes, the overarching behaviors perfectly mirrored theoretical expectations documented in modern self-supervised literature, alongside several interesting nuances observed during training. First, the SimCLR model successfully learned highly discriminative features without relying on external labels. The learning curves revealed that the K-NN accuracy surged rapidly during the first 50 epochs before gradually plateauing, which is a classic stabilization trajectory for contrastive representation spaces. We also confirmed that

continuous kNN monitoring is absolutely vital; the raw NT-Xent loss curve is entirely disconnected from actual representation quality (as definitively proven by our mathematical analysis of the temperature ablation). One slightly unexpected observation was the Batch Size ablation. While literature emphasizes that SimCLR requires massive batch sizes to provide enough negative samples to prevent trivial solutions, our experiments on CIFAR-10 showed remarkable resilience even down to a batch size of 32. This observation suggests that for a low-diversity dataset like CIFAR-10, smaller batches still capture sufficient negative distribution variance to form distinct, linearly separable feature clusters.

4.2 Domain Gap and Dataset Diversity Constraints

Factors affecting the results and Transferability:

The most profound finding of this research was the Transfer Learning experiment, which distinctly highlighted the impact of the "Domain Gap" and "Dataset Diversity" factors. While Supervised Learning utterly dominated on CIFAR-10 (92.26%), it severely overfitted to those 10 specific macroscopic semantic classes (e.g., distinguishing a dog from an airplane). When tested on CIFAR-100, the SL backbone struggled significantly (43.38%) because its objective function had strictly trained it to treat fine-grained details (e.g., the specific shape of a leaf, the texture of an insect, or the breed of a mammal) as irrelevant noise during CIFAR-10 training. In sharp contrast, SimCLR was forced to learn general-purpose, instance-level discriminative features simply to separate heavy image augmentations. Because it learned "what makes an image unique" rather than "what makes this a dog", its representations were much richer and transferred significantly better to CIFAR-100 (45.43%).

However, it is critically important to contextualize that 45.43% is still objectively low for CIFAR-100 classification. This limitation is fundamentally restricted by the characteristics of the pre-training dataset itself. True foundation models (like CLIP or DINO) derive their immense power from massive, highly varied datasets (like ImageNet or LAION). Because CIFAR-10 possesses an extremely narrow scope of visual diversity, it acts as a permanent ceiling for transferability, regardless of how well the contrastive objective is mathematically optimized.

4.3 Future Trajectories and Remaining Questions

What I Learned and Extrapolations for Future Work:

Through these rigorous experiments, I learned that while Supervised Learning provides the fastest mathematical path to high accuracy on a highly specific target task, it produces inherently brittle feature manifolds compared to SSL when shifted to novel domains. I also learned that while contrastive loss gracefully minimizes the distance between augmented views, evaluating that loss alone in a vacuum is meaningless without continuously measuring downstream utility.

If more computational time and resources were available, I would systematically test

the impact of different individual augmentation types (e.g., performing an ablation where color jittering is completely removed while spatial cropping remains). SimCLR is known to rely heavily on the precise composition of specific augmentations to prevent the network from discovering trivial "shortcuts" (like exploiting chromatic aberration). Furthermore, I would explore the impact of drastically increasing the training duration (e.g., scaling to 1000 or 2000 epochs) to determine if SimCLR's asymptotic performance limit could eventually surpass Supervised performance directly on CIFAR-10. Finally, I would investigate Semi-Supervised Learning regimes—specifically, utilizing the SSL pre-trained backbone and fine-tuning it with only 1% or 10% of the available CIFAR-10 labels—to fully demonstrate the label-efficiency promised by foundation models.

5 References

1. Ting Chen, Simon Kornblith, Mohammad Norouzi, Geoffrey Hinton (2020). A Simple Framework for Contrastive Learning of Visual Representations. *arXiv preprint arXiv:2002.05709*.

A Source Code Listings

A.1 run_all.py

```
import subprocess
import sys

def run_cmd(cmd):
    print(f"\n===== RUNNING: {cmd} =====\n")
    result = subprocess.run(cmd, shell=True)
    if result.returncode != 0:
        print(f"Command failed with exit code {result.returncode}")
        sys.exit(result.returncode)

def main():
    # 1. Train Supervised Baseline (from scratch, 100 epochs)
    run_cmd("python train_supervised.py --epochs 100 --batch_size 256 --save_name sl_baseline.pth")

    # 2. Train SimCLR Baseline (using batch_size 256, 200 epochs)
    run_cmd("python train_simclr.py --epochs 200 --batch_size 256 --save_name simclr_baseline.pth")

    # 3. Linear Probing on SimCLR Baseline (100 epochs as requested in PDF)
    run_cmd("python train_linprobe.py --epochs 100 --pretrained simclr_baseline.pth --save_name linprobe_simclr.pth")

    # 4. Lower-Bound Random Baseline
    run_cmd("python train_linprobe.py --epochs 100 --random --save_name linprobe_random_baseline.pth")

    # 5. Temperature Ablations (200 epochs for fair comparison with baseline)
    run_cmd("python train_simclr.py --epochs 200 --batch_size 256 --temperature 5.0 --save_name simclr_temp5.pth")
    run_cmd("python train_simclr.py --epochs 200 --batch_size 256 --temperature 0.1 --save_name simclr_temp01.pth")

    # 6. Batch Size Ablations
    run_cmd("python train_simclr.py --epochs 200 --batch_size 128 --save_name simclr_bs128.pth")
    run_cmd("python train_simclr.py --epochs 200 --batch_size 64 --save_name simclr_bs64.pth")
    run_cmd("python train_simclr.py --epochs 200 --batch_size 32 --save_name simclr_bs32.pth")

    # 7. Projector Ablations
    # Train without projector
```

```

run_cmd("python train_simclr.py --epochs 200 --batch_size 256 --
        save_name simclr_noproj.pth --no_projector")
run_cmd("python train_linprobe.py --epochs 100 --pretrained
        simclr_noproj.pth --save_name linprobe_simclr_noproj.pth")
# Linprobe WITH projector outputs from baseline
run_cmd("python train_linprobe.py --epochs 100 --pretrained
        simclr_baseline.pth --save_name linprobe_simclr_projout.pth --
        use_projector")

# 8. Transfer Learning on CIFAR-100
run_cmd("python train_linprobe.py --epochs 100 --random --dataset
        cifar100 --save_name linprobe_random_cifar100.pth")
run_cmd("python train_linprobe.py --epochs 100 --pretrained_sl
        sl_baseline.pth --dataset cifar100 --save_name
        linprobe_sl_cifar100.pth")
run_cmd("python train_linprobe.py --epochs 100 --pretrained
        simclr_baseline.pth --dataset cifar100 --save_name
        linprobe_simclr_cifar100.pth")

# 9. Plot everything
run_cmd("python plot_results.py")
print("ALL EXPERIMENTS COMPLETED!")

if __name__ == "__main__":
    main()

```

A.2 dataset.py

```

import torchvision.transforms as transforms

class SimCLRTransform:
    def __init__(self, size=32):
        # Hyperparameters commonly used for CIFAR
        s = 1
        color_jitter = transforms.ColorJitter(0.8 * s, 0.8 * s, 0.8 * s
        , 0.2 * s)
        self.train_transform = transforms.Compose([
            transforms.RandomResizedCrop(size=size),
            transforms.RandomHorizontalFlip(),
            transforms.RandomApply([color_jitter], p=0.8),
            transforms.RandomGrayscale(p=0.2),
            transforms.ToTensor(),
            # CIFAR-10 stats
            transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023,
            0.1994, 0.2010))
        ])

    def __call__(self, x):
        return self.train_transform(x), self.train_transform(x)

```

```

def get_cifar10_transforms():
    test_transform = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023, 0.1994,
            0.2010))
    ])
    train_sl_transform = transforms.Compose([
        transforms.RandomCrop(32, padding=4),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(),
        transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023, 0.1994,
            0.2010))
    ])
    return SimCLRTransform(size=32), train_sl_transform, test_transform

```

A.3 models.py

```

import torch
import torch.nn as nn
import torchvision.models as models

class ModifiedResNet18(nn.Module):
    def __init__(self):
        super().__init__()
        resnet = models.resnet18(weights=None)

        # Modify conv1: 3x3 kernel, stride=1, padding=1
        resnet.conv1 = nn.Conv2d(3, 64, kernel_size=3, stride=1,
            padding=1, bias=False)
        # Modify maxpool to identity
        resnet.maxpool = nn.Identity()

        # Remove fc layer to get 512 dim output (before flatten, it's
            global avg pooling)
        self.encoder = nn.Sequential(*list(resnet.children())[:-1])

    def forward(self, x):
        h = self.encoder(x)
        h = h.view(h.shape[0], -1) # Flatten to (B, 512)
        return h

class ProjectorHead(nn.Module):
    def __init__(self, in_features=512, hidden_features=512,
        out_features=128):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(in_features, hidden_features),
            nn.BatchNorm1d(hidden_features),

```

```

        nn.ReLU(inplace=True),
        nn.Linear(hidden_features, out_features)
    )

    def forward(self, x):
        return self.net(x)

class SimCLRNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.backbone = ModifiedResNet18()
        self.projector = ProjectorHead()

    def forward(self, x):
        h = self.backbone(x)
        z = self.projector(h)
        return h, z

class LinearClassifier(nn.Module):
    def __init__(self, in_features=512, num_classes=10):
        super().__init__()
        self.fc = nn.Linear(in_features, num_classes)

    def forward(self, x):
        return self.fc(x)

```

A.4 loss.py

```

import torch
import torch.nn as nn
import torch.nn.functional as F

class NTXentLoss(nn.Module):
    def __init__(self, temperature=0.5):
        super().__init__()
        self.temperature = temperature
        self.criterion = nn.CrossEntropyLoss(reduction="mean")

    def forward(self, z_i, z_j):
        """
        z_i and z_j are B x 128 normalized feature maps
        """
        batch_size = z_i.shape[0]
        device = z_i.device

        z_i = F.normalize(z_i, dim=1)
        z_j = F.normalize(z_j, dim=1)

        z = torch.cat([z_i, z_j], dim=0) # 2N x 128

```

```

# Pairwise cosine similarity matrix
sim_matrix = torch.matmul(z, z.T) # 2N x 2N
sim_matrix = sim_matrix / self.temperature

# Mask out self-similarity
mask = torch.eye(2 * batch_size, dtype=torch.bool).to(device)
sim_matrix.masked_fill_(mask, -9e15)

# Labels: the mate of i is i + N, mate of i + N is i
positives = torch.cat([torch.arange(batch_size, 2*batch_size),
    torch.arange(0, batch_size)]).to(device)

loss = self.criterion(sim_matrix, positives)
return loss

```

A.5 utils.py

```

import torch
import torch.nn.functional as F

def knn_predict(feature, feature_bank, feature_labels, classes, knn_k,
    knn_t):
    # compute cos similarity between each feature vector and feature
    # bank ---> [B, N]
    sim_matrix = torch.mm(feature, feature_bank)
    # [B, K]
    sim_weight, sim_indices = sim_matrix.topk(k=knn_k, dim=-1)
    # [B, K]
    sim_labels = torch.gather(feature_labels.expand(feature.size(0),
        -1), dim=-1, index=sim_indices)
    sim_weight = (sim_weight / knn_t).exp()

    # counts for each class
    one_hot_label = torch.zeros(feature.size(0) * knn_k, classes,
        device=sim_labels.device)
    # [B*K, C]
    one_hot_label = one_hot_label.scatter(dim=-1, index=sim_labels.view
        (-1, 1), value=1.0)
    # weighted score ---> [B, C]
    pred_scores = torch.sum(one_hot_label.view(feature.size(0), -1,
        classes) * sim_weight.unsqueeze(dim=-1), dim=1)

    pred_labels = pred_scores.argsort(dim=-1, descending=True)
    return pred_labels

def knn_monitor(net, memory_data_loader, test_data_loader, device, k
    =20, t=0.1, num_classes=10):

```

```

net.eval()
classes = num_classes
total_top1 = 0
total_num = 0
feature_bank = []

with torch.no_grad():
    # generate feature bank
    for data, target in memory_data_loader:
        # For SimCLR, memory loader returns (data, target)
        feature = net(data.to(device, non_blocking=True))
        feature = F.normalize(feature, dim=1)
        feature_bank.append(feature)

    # [D, N]
    feature_bank = torch.cat(feature_bank, dim=0).t().contiguous()

    # [N]
    feature_labels = torch.tensor(memory_data_loader.dataset.
        targets, device=device)

    # loop test data to predict the label by weighted knn search
    for data, target in test_data_loader:
        data, target = data.to(device, non_blocking=True), target.
            to(device, non_blocking=True)
        feature = net(data)
        feature = F.normalize(feature, dim=1)

        pred_labels = knn_predict(feature, feature_bank,
            feature_labels, classes, k, t)

        total_num += data.size(0)
        total_top1 += (pred_labels[:, 0] == target).float().sum().
            item()

return total_top1 / total_num * 100

```

A.6 train_simclr.py

```

import os
import tempfile
import torch
import torch.optim as optim
from torch.utils.data import DataLoader
from torchvision.datasets import CIFAR10
from tqdm import tqdm
import json

from dataset import get_cifar10_transforms

```



```

from models import SimCLRNet
from loss import NTXentLoss
from utils import knn_monitor

def train_simclr(batch_size=256, epochs=200, lr=3e-4, wd=1e-6,
    temperature=0.5, save_name="simclr_model.pth", no_projector=False):
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    simclr_transform, _, test_transform = get_cifar10_transforms()

    train_dataset = CIFAR10(root='./data', train=True, transform=
        simclr_transform, download=True)
    train_loader = DataLoader(train_dataset, batch_size=batch_size,
        shuffle=True, num_workers=4, pin_memory=True, drop_last=True)

    memory_dataset = CIFAR10(root='./data', train=True, transform=
        test_transform, download=True)
    memory_loader = DataLoader(memory_dataset, batch_size=batch_size,
        shuffle=False, num_workers=4, pin_memory=True)

    test_dataset = CIFAR10(root='./data', train=False, transform=
        test_transform, download=True)
    test_loader = DataLoader(test_dataset, batch_size=batch_size,
        shuffle=False, num_workers=4, pin_memory=True)

    model = SimCLRNet().to(device)
    optimizer = optim.Adam(model.parameters(), lr=lr, weight_decay=wd)
    criterion = NTXentLoss(temperature=temperature)

    logs = {"loss": [], "knn_acc": []}

    for epoch in range(1, epochs + 1):
        model.train()
        total_loss = 0
        pbar = tqdm(train_loader, desc=f"Epoch {epoch}/{epochs}")
        for (img1, img2), _ in pbar:
            img1, img2 = img1.to(device), img2.to(device)

            optimizer.zero_grad()
            h1, z1 = model(img1)
            h2, z2 = model(img2)

            if no_projector:
                loss = criterion(h1, h2)
            else:
                loss = criterion(z1, z2)
            loss.backward()
            optimizer.step()

```

```

        total_loss += loss.item()
        pbar.set_postfix({"Loss": loss.item()})

    avg_loss = total_loss / len(train_loader)
    logs["loss"].append(avg_loss)
    print(f"Epoch {epoch} | Avg Loss: {avg_loss:.4f}")

    if epoch % 5 == 0 or epoch == 1 or epoch == epochs:
        knn_acc = knn_monitor(model.backbone, memory_loader,
                               test_loader, device, k=20)
        logs["knn_acc"].append((epoch, knn_acc))
        print(f"Epoch {epoch} | kNN Monitor Accuracy: {knn_acc:.2f}
              }%")

    torch.save(model.state_dict(), save_name)
    with open(save_name.replace(".pth", "_logs.json"), "w") as f:
        json.dump(logs, f)

if __name__ == "__main__":
    import argparse
    parser = argparse.ArgumentParser()
    parser.add_argument("--batch_size", type=int, default=256)
    parser.add_argument("--epochs", type=int, default=200)
    parser.add_argument("--temperature", type=float, default=0.5)
    parser.add_argument("--save_name", type=str, default="
        simclr_baseline.pth")
    parser.add_argument("--no_projector", action="store_true")
    args = parser.parse_args()

    train_simclr(batch_size=args.batch_size, epochs=args.epochs,
                  temperature=args.temperature, save_name=args.save_name,
                  no_projector=args.no_projector)

```

A.7 train_linprobe.py

```

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
from torchvision.datasets import CIFAR10, CIFAR100
from tqdm import tqdm
import json

from dataset import get_cifar10_transforms
from models import SimCLRNet, LinearClassifier, ModifiedResNet18

def train_linprobe(pretrained_path=None, pretrained_sl_path=None,
                   batch_size=256, epochs=100, lr=1e-3, wd=1e-6, save_name="
    linprobe_model.pth", is_random_baseline=False, dataset_name='cifar10

```

```

', use_projector=False):
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    _, train_transform, test_transform = get_cifar10_transforms()

    if dataset_name == 'cifar100':
        train_dataset = CIFAR100(root='./data', train=True, transform=
            train_transform, download=True)
        test_dataset = CIFAR100(root='./data', train=False, transform=
            test_transform, download=True)
        num_classes = 100
    else:
        train_dataset = CIFAR10(root='./data', train=True, transform=
            train_transform, download=True)
        test_dataset = CIFAR10(root='./data', train=False, transform=
            test_transform, download=True)
        num_classes = 10

    train_loader = DataLoader(train_dataset, batch_size=batch_size,
        shuffle=True, num_workers=4, pin_memory=True)
    test_loader = DataLoader(test_dataset, batch_size=batch_size,
        shuffle=False, num_workers=4, pin_memory=True)

    if not is_random_baseline and pretrained_path:
        simclr_model = SimCLRNet()
        simclr_model.load_state_dict(torch.load(pretrained_path))
        if use_projector:
            backbone = simclr_model
            in_features = 128
        else:
            backbone = simclr_model.backbone
            in_features = 512
    elif pretrained_sl_path:
        sl_model = nn.Sequential(ModifiedResNet18(), LinearClassifier(
            in_features=512, num_classes=10))
        sl_model.load_state_dict(torch.load(pretrained_sl_path))
        backbone = sl_model[0]
        in_features = 512
    else:
        backbone = ModifiedResNet18()
        in_features = 512

    backbone = backbone.to(device)
    # Freeze backbone
    for param in backbone.parameters():
        param.requires_grad = False

    classifier = LinearClassifier(in_features=in_features, num_classes=
        num_classes).to(device)

```

```

optimizer = optim.Adam(classifier.parameters(), lr=lr, weight_decay
                        =wd)
criterion = nn.CrossEntropyLoss()

logs = {"loss": [], "test_acc": []}

for epoch in range(1, epochs + 1):
    classifier.train()
    total_loss = 0
    pbar = tqdm(train_loader, desc=f"Epoch {epoch}/{epochs}")
    for imgs, targets in pbar:
        imgs, targets = imgs.to(device), targets.to(device)

        with torch.no_grad():
            features = backbone(imgs)
            if isinstance(features, tuple):
                features = features[1] # Use projector output

        optimizer.zero_grad()
        outputs = classifier(features)
        loss = criterion(outputs, targets)
        loss.backward()
        optimizer.step()

        total_loss += loss.item()

    avg_loss = total_loss / len(train_loader)
    logs["loss"].append(avg_loss)

    # Eval
    classifier.eval()
    correct = 0
    total = 0
    with torch.no_grad():
        for imgs, targets in test_loader:
            imgs, targets = imgs.to(device), targets.to(device)
            features = backbone(imgs)
            if isinstance(features, tuple):
                features = features[1]
            outputs = classifier(features)
            _, predicted = outputs.max(1)
            total += targets.size(0)
            correct += predicted.eq(targets).sum().item()

    acc = 100. * correct / total
    logs["test_acc"].append((epoch, acc))
    print(f"Epoch {epoch} | Loss: {avg_loss:.4f} | Test Acc: {acc
        :.2f}%")

```

```

torch.save(classifier.state_dict(), save_name)
with open(save_name.replace(".pth", "_logs.json"), "w") as f:
    json.dump(logs, f)

if __name__ == "__main__":
    import argparse
    parser = argparse.ArgumentParser()
    parser.add_argument("--pretrained", type=str, default=None)
    parser.add_argument("--pretrained_sl", type=str, default=None)
    parser.add_argument("--epochs", type=int, default=100)
    parser.add_argument("--random", action="store_true")
    parser.add_argument("--save_name", type=str, default=None)
    parser.add_argument("--dataset", type=str, default="cifar10")
    parser.add_argument("--use_projector", action="store_true")
    args = parser.parse_args()

    if args.random:
        save_name = args.save_name if args.save_name else "
            linprobe_random_baseline.pth"
        train_linprobe(is_random_baseline=True, save_name=save_name,
            epochs=args.epochs, dataset_name=args.dataset)
    elif args.pretrained_sl:
        save_name = args.save_name if args.save_name else "
            linprobe_sl_baseline.pth"
        train_linprobe(pretrained_sl_path=args.pretrained_sl, save_name
            =save_name, epochs=args.epochs, dataset_name=args.dataset)
    else:
        save_name = args.save_name if args.save_name else "
            linprobe_simclr.pth"
        train_linprobe(pretrained_path=args.pretrained, save_name=
            save_name, epochs=args.epochs, dataset_name=args.dataset,
            use_projector=args.use_projector)

```

A.8 plot_results.py

```

import json
import matplotlib.pyplot as plt
import os

def plot_simclr(log_file, ax_loss=None, ax_acc=None, label="SimCLR"):
    if not os.path.exists(log_file): return False
    with open(log_file, "r") as f:
        logs = json.load(f)

    losses = logs["loss"]
    epochs_loss = range(1, len(losses) + 1)

    accs = logs["knn_acc"]
    epochs_acc = [x[0] for x in accs]

```

```

acc_vals = [x[1] for x in accs]

if ax_loss: ax_loss.plot(epochs_loss, losses, label=label)
if ax_acc: ax_acc.plot(epochs_acc, acc_vals, label=label, marker='o')
return True

def plot_supervised(log_file, ax_loss=None, ax_acc=None, label="Supervised"):
    if not os.path.exists(log_file): return False
    with open(log_file, "r") as f:
        logs = json.load(f)

    losses = logs["loss"]
    epochs_loss = range(1, len(losses) + 1)

    accs = logs["test_acc"]
    epochs_acc = [x[0] for x in accs]
    acc_vals = [x[1] for x in accs]

    if ax_loss: ax_loss.plot(epochs_loss, losses, label=label)
    if ax_acc: ax_acc.plot(epochs_acc, acc_vals, label=label)
    return True

def get_final_acc(log_file):
    if not os.path.exists(log_file): return 0.0
    with open(log_file, "r") as f:
        logs = json.load(f)
    return logs["test_acc"][-1][1]

def main():
    # 1. Main loss and kNN curve for SimCLR Baseline
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))
    plot_simclr("simclr_baseline_logs.json", ax1, ax2, "Baseline SimCLR
        (bs=256, temp=0.5)")
    ax1.set_title("Training NT-Xent Loss")
    ax1.set_xlabel("Epochs")
    ax1.set_ylabel("Loss")
    ax1.legend()
    ax2.set_title("kNN Monitor Accuracy (k=20)")
    ax2.set_xlabel("Epochs")
    ax2.set_ylabel("Accuracy (%)")
    ax2.legend()
    plt.tight_layout()
    plt.savefig("simclr_learning_curves.png", dpi=300)
    print("Saved simclr_learning_curves.png")

    # 2. Supervised Learning loss and accuracy curve
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))

```

```

plot_supervised("sl_baseline_logs.json", ax1, ax2, "Supervised
    ResNet-18")
ax1.set_title("Training Cross-Entropy Loss")
ax1.set_xlabel("Epochs")
ax1.set_ylabel("Loss")
ax1.legend()
ax2.set_title("Test Set Accuracy")
ax2.set_xlabel("Epochs")
ax2.set_ylabel("Accuracy (%)")
ax2.legend()
plt.tight_layout()
plt.savefig("supervised_learning_curves.png", dpi=300)
print("Saved supervised_learning_curves.png")

# 3. Baseline Comparisons Bar Chart (CIFAR-10)
acc_rand = get_final_acc("linprobe_random_baseline_logs.json")
acc_simclr = get_final_acc("linprobe_simclr_logs.json")
acc_sl = get_final_acc("sl_baseline_logs.json")

plt.figure(figsize=(8, 6))
bars = plt.bar(["Random Backbone", "SimCLR SSL", "Supervised"], [
    acc_rand, acc_simclr, acc_sl], color=['gray', 'blue', 'green'])
plt.title("CIFAR-10 Classification Accuracy")
plt.ylabel("Accuracy (%)")
for bar in bars:
    yval = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2, yval + 1, f'{yval:.2f}
        }%', ha='center', va='bottom')
plt.tight_layout()
plt.savefig("baseline_comparisons.png", dpi=300)
print("Saved baseline_comparisons.png")

# 4. Temperature Ablations Plot
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))
plot_simclr("simclr_baseline_logs.json", ax1, ax2, "temp=0.5 (
    Baseline)")
plot_simclr("simclr_temp5_logs.json", ax1, ax2, "temp=5.0")
plot_simclr("simclr_temp01_logs.json", ax1, ax2, "temp=0.1")
ax1.set_title("Loss with Temperature Variations")
ax1.set_xlabel("Epochs")
ax1.set_ylabel("Loss")
ax1.legend()
ax2.set_title("kNN Accuracy with Temperature Variations")
ax2.set_xlabel("Epochs")
ax2.set_ylabel("Accuracy (%)")
ax2.legend()
plt.tight_layout()
plt.savefig("ablation_temperature.png", dpi=300)
print("Saved ablation_temperature.png")

```

```

# 5. Batch Size Ablations Plot
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))
plot_simclr("simclr_baseline_logs.json", ax1, ax2, "BS=256 (
    Baseline)")
plot_simclr("simclr_bs128_logs.json", ax1, ax2, "BS=128")
plot_simclr("simclr_bs64_logs.json", ax1, ax2, "BS=64")
plot_simclr("simclr_bs32_logs.json", ax1, ax2, "BS=32")
ax1.set_title("Loss with Batch Size Variations")
ax1.set_xlabel("Epochs")
ax1.set_ylabel("Loss")
ax1.legend()
ax2.set_title("kNN Accuracy with Batch Size Variations")
ax2.set_xlabel("Epochs")
ax2.set_ylabel("Accuracy (%)")
ax2.legend()
plt.tight_layout()
plt.savefig("ablation_batch_size.png", dpi=300)
print("Saved ablation_batch_size.png")

# 6. Projector Ablations Bar Chart
acc_simclr = get_final_acc("linprobe_simclr_logs.json") # Baseline
backbone
acc_noproj = get_final_acc("linprobe_simclr_noproj_logs.json") #
Trained without proj
acc_projout = get_final_acc("linprobe_simclr_projout_logs.json") #
Linprobe on proj outputs

plt.figure(figsize=(10, 6))
bars = plt.bar(["Standard (Encoder)", "Trained w/o Projector", "
    Linear Probe on Projector"],
               [acc_simclr, acc_noproj, acc_projout], color=['blue',
                   , 'orange', 'purple'])
plt.title("Projector Ablations on CIFAR-10")
plt.ylabel("Accuracy (%)")
for bar in bars:
    yval = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2, yval + 1, f'{yval:.2f
        }%', ha='center', va='bottom')
plt.tight_layout()
plt.savefig("ablation_projector.png", dpi=300)
print("Saved ablation_projector.png")

# 7. Transfer Learning (CIFAR-100) Bar Chart
acc_t_rand = get_final_acc("linprobe_random_cifar100_logs.json")
acc_t_sl = get_final_acc("linprobe_sl_cifar100_logs.json")
acc_t_simclr = get_final_acc("linprobe_simclr_cifar100_logs.json")

plt.figure(figsize=(8, 6))
bars = plt.bar(["Random Backbone", "Supervised (CIFAR-10)", "SimCLR
    (CIFAR-10)"],

```



```

        [acc_t_rand, acc_t_sl, acc_t_simclr], color=['gray',
            'green', 'blue'])
plt.title("Transfer Learning Accuracy on CIFAR-100")
plt.ylabel("Accuracy (%)")
for bar in bars:
    yval = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2, yval + 0.5, f'{yval
        :.2f}%', ha='center', va='bottom')
plt.tight_layout()
plt.savefig("transfer_learning_cifar100.png", dpi=300)
print("Saved transfer_learning_cifar100.png")

if __name__ == "__main__":
    main()

```